

1. Описание условия задания

Создать таблицу, содержащую не менее 40-ка записей (тип – запись с вариантами (объединениями)). Упорядочить данные в ней по возрастанию ключей, двумя алгоритмами сортировки, где ключ – любое невариантное поле (по выбору программиста), используя: а) саму таблицу, б) массив ключей. Возможность добавления и удаления записей в ручном режиме, просмотр таблицы, просмотр таблицы в порядке расположения таблицы ключей обязательна. Осуществить поиск информации по варианту.

Ввести список стран, содержащий:

- Название стран;
- Столицу;
- Материк;
- Необходимость наличия визы;
- Время полета до страны;
- Минимальную стоимость отдыха;
- Основной вид туризма:
 - 1. Экскурсионный:
 - а. Количество объектов;
 - б. Основной вид объектов (природа, искусство, история);
 - 2. пляжный:
 - а. Основной сезон;
 - б. Температура воздуха и воды;
 - 3. Спортивный:
 - а. Вид спорта (горные лыжи, серфинг, восхождения)

Вывести список стран на выбранном материке, где можно заняться указанным видом спорта, со стоимостью отдыха меньше указанной.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные: файл, содержащий информацию о странах.

Ограничения:

- Строковое поле ≤ 20 символов;
- Необходимость наличия визы – 0 либо 1;
- Время полёта (в часах) – целое неотрицательное число;
- Минимальная стоимость отдыха – целое неотрицательное число;
- Вид туризма – либо 0, либо 1, либо 2;
- Количество объектов -- целое неотрицательное число;
- Тип объекта – либо 0, либо 1, либо 2;
- Основной сезон – либо 0, либо 1, либо 2, либо 3;
- Температура воздуха – целое число;
- Температура воды – целое число;
- Вид спорта -- либо 0, либо 1, либо 2;
- Количество стран ≤ 1000 .

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- Некорректный файл;
- Некорректный ввод;
- Некорректные входные данные;
- Пустой файл;
- Переполнение массива стран.

Возможные действия, реализуемой программой:

0. Выход из программы;
1. Загрузка данных из файла;
2. Вывод таблицы стран;
3. Добавление страны;
4. Удаление страны;
5. Поиск стран по виду спорта;
6. Сортировка исходной таблицы стран методом пузырька;
7. Сортировка таблицы ключей методом пузырька;
8. Быстрая сортировка исходной таблицы стран;
9. Быстрая сортировка таблицы ключей;

- 10.** Вывод исходной таблицы с помощью таблицы ключей;
- 11.** Сравнение эффективности алгоритмов;
- 12.** Вывод таблицы ключей;
- 13.** Инициализация таблицы ключей;
- 14.** Сохранение данных в файл.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мною были реализованы следующие структуры данных:

Виды туризма
<pre>typedef enum { EXCURSION, BEACH, SPORT } tourism_t;</pre>
Виды объектов
<pre>typedef enum { NATURE, ART, HISTORY } object_t;</pre>
Сезоны
<pre>typedef enum { SUMMER, WINTER, SPRING, AUTUMN } season_t;</pre>
Виды спорта
<pre>typedef enum { SKIING, SURFING, CLIMBING } sport_t;</pre>

Данные поля основной структуры были реализованы с помощью перечисляемого типа, так что поля данных типов равны соответственно: 0, 1, 2 и [3].

Основная структура:

Страна
<pre>typedef struct { char name[MAX_STRING_LEN]; char capital[MAX_STRING_LEN]; char continent[MAX_STRING_LEN]; int visa_required; int flight_time; int min_cost; tourism_t tourism; union { struct { int objects_count; object_t object_type; } excursion; struct { season_t season; } season; }; };</pre>

```

        int air_temp;
        int water_temp;
    } beach;
    struct {
        sport_t sport_type;
    } sport;
} tourism_details;
} country t;

```

Поля основной структуры:

- **name** – название страны (MAX_STRING_LEN == 20);
- **capital** – название столицы;
- **continent** – название континента;
- **visa_required** – необходимость визы;
- **flight_time** – длительность полёта;
- **min_cost** – минимальная стоимость;
- **tourism** – вид туризма;
- **object_count** – количество объектов для туризма;
- **object_type** – тип объектов;
- **season** – основной сезон;
- **air_temp** – температура воздуха;
- **water_temp** – температура воды;
- **sport_type** – вид спорта.

Структура ключа:

Ключ

```

typedef struct {
    size_t index;
    int key; // ключ - минимальная стоимость
} key t;

```

Поля структуры ключей:

- **index** – индекс определённой записи;
- **key** – значение ключевого поля определённой страны (была выбрана минимальная стоимость).

4. Алгоритмы

Основной алгоритм программы:

1. Вывод данных о структуре страны;
2. Вывод меню;
3. Считывание выбора пользователя и проверка его корректности;
4. Выполнение соответствующего действия;
5. Либо снова вывод меню с ожиданием выбора, либо завершение программы с сообщением об ошибке.

Загрузка данных из файла:

1. Ввод названия файла;
2. Открытие файла с проверками на ошибки;
3. Чтение данных из файла с проверками на ошибки;
4. Проверка на переполнение массива;
5. Закрытие файла.

Добавление записи в массив:

1. Проверка на переполнение массива;
2. Ввод каждого поля с проверками на ошибки;
3. Добавление новой записи в массив;
4. Увеличение счётчика элементов массива.

Удаление записи из массива:

1. Проверка наличия элементов в массиве;
2. Ввод названия страны для удаления и проверка названия на корректность;
3. Обход массива с поиском страны с соответствующим названием;
4. В случае нахождения -- циклический сдвиг элементов;
5. В случае отсутствия результата поиска -- сообщение об отсутствии результата поиска.

Поиск определённых записей:

1. Чтение полей: материк, вид спорта, минимальная стоимость;
2. Обход массива с поиском страны по заданным критериям и минимальной стоимостью меньше введённой;
3. Вывод найденных стран или сообщение об отсутствии результата поиска.

Сравнение эффективности алгоритмов:

1. Создание дополнительных копий массива стран;
2. Цикл из 5-ти итераций для более точного результата:
 - a. Пузырьковая сортировка исходной таблицы;
 - b. Быстрая сортировка исходной таблицы;

- c. Инициализация таблицы ключей;
 - d. Пузырьковая сортировка таблицы ключей;
 - e. Инициализация таблицы ключей;
 - f. Быстрая сортировка таблицы ключей;
3. Вычисление средних результатов замеров времени;
 4. Вывод результата по времени;
 5. Вычисление занимаемой оперативной памяти от способа взаимодействия;
 6. Вывод результата по памяти.

Реализуемые функции:

- // Функция вывода главного меню
- 1. **void print_menu(void)**
-
- // Функции вывода одной страны
- 2. **void print_country(const country_t *c)**
-
- // Функция вывода шапки таблицы
- 3. **void print_head(void)**
-
- // Функция вывода таблицы стран
- 4. **void print_data(const country_t countries[], const size_t number_countries)**
-
- // Функция вывода таблицы ключей
- 5. **void print_key_table(const key_t key_table[], const size_t number_countries)**
-
- // Функция вывода таблицы стран через таблицу ключей
- 6. **void print_by_key(const key_t key_table[], const country_t countries[], const size_t number_countries)**
-
- // Функция вывода информации о стране
- 7. **void print_structure_info(void)**
-
- // Функция для загрузки первоначального массива стран из файла
- 8. **int load_data(country_t countries[], size_t *number_countries)**

// Функция для сохранения данных в файл

9. int save_data(const country_t countries[], const size_t number_countries)

// Функция для добавления страны

10. int add_record(country_t countries[], size_t *number_countries)

// Функция для удаления страны

11. int delete_country(country_t countries[], size_t *number_countries)

// Функция для поиска по спортивному туризму

12. int search_by_sport(const country_t countries[], const size_t number_countries)

// Инициализация таблицы ключей

13. void initialize_key_table(key_t key_table[], const country_t countries[], const size_t number_countries)

// Пузырьковая сортировка таблицы ключей

14. void bubble_sort_key(key_t key_table[], const size_t number_countries)

// Быстрая сортировка таблицы ключей

15. void quick_sort_key(key_t key_table[], int low, int high)

// Пузырьковая сортировка исходной таблицы

16. void bubble_sort(country_t countries[], const size_t number_countries)

// Быстрая сортировка исходной таблицы

17. void quick_sort(country_t countries[], int low, int high)

// Тестирование алгоритмов сортировки

18. void test_sorting(const country_t countries[], const size_t number_countries)

5. Тесты

Позитивные тесты:

1. Чтение файла, содержащего одну запись;
2. Чтение файла, содержащего 1000 записей;
3. Попытка взаимодействия с массивом стран без загрузки массива из файла;
4. Вывод исходной таблицы;
5. Добавление страны;
6. Удаление существующей страны;
7. Удаление несуществующей страны;
8. Поиск единственной страны;
9. Поиск нескольких стран;
10. Поиск несуществующих стран;
11. Сортировка пузырьком одного элемента исходной таблицы;
12. Сортировка пузырьком нескольких стран, находящихся по убыванию стоимости;
13. Сортировка пузырьком нескольких стран, находящихся по возрастанию стоимости;
14. Сортировка пузырьком нескольких стран с одинаковой стоимостью;
15. Сортировка пузырьком 1000 стран со случайной стоимостью;
16. Быстрая сортировка одного элемента исходной таблицы;
17. Быстрая сортировка нескольких стран, находящихся по убыванию стоимости;
18. Быстрая сортировка нескольких стран, находящихся по возрастанию стоимости;
19. Быстрая сортировка нескольких стран с одинаковой стоимостью;
20. Быстрая сортировка 1000 стран со случайной стоимостью;
21. Сортировка пузырьком одного элемента таблицы ключей;
22. Сортировка пузырьком нескольких ключей, находящихся по убыванию стоимости;
23. Сортировка пузырьком нескольких ключей, находящихся по возрастанию стоимости;
24. Сортировка пузырьком нескольких ключей с одинаковой стоимостью;
25. Сортировка пузырьком 1000 ключей со случайной стоимостью;
26. Быстрая сортировка одного элемента таблицы ключей;
27. Быстрая сортировка нескольких ключей, находящихся по убыванию стоимости;
28. Быстрая сортировка нескольких ключей, находящихся по возрастанию стоимости;
29. Быстрая сортировка нескольких ключей с одинаковой стоимостью;
30. Быстрая сортировка 1000 ключей со случайной стоимостью;
31. Вывод исходной таблицы через таблицу ключей;

- | |
|---|
| <ul style="list-style-type: none">32. Вывод неинициализированной таблицы ключей;33. Вывод инициализированной таблицы ключей;34. Инициализация таблицы ключей с одним элементом;35. Инициализация таблицы ключей с 1000 элементами;36. Сравнение эффективности алгоритмов с 1 элементом;37. Сравнение эффективности алгоритмов с 1000 элементами;38. Сохранение массива стран в корректный файл; |
|---|

Негативные тесты:
1. Пустой исходный файл; 2. Невалидный исходный файл; 3. Невалидный выбор меню; 4. Невалидный ввод при добавлении страны; 5. Переполнение массива стран при добавлении страны; 6. Переполнение массива стран при чтении исходного файла; 7. Невалидный ввод при удалении страны; 8. Невалидный ввод при поиске стран;

6. Оценка эффективности

Сравнение результатов по времени:

С каждым из видов сортировки пять раз выполняется замер времени с несколькими размерами массива. В таблицу вписывается усредненное среди этих пяти замеров значение.

Размер массива	Сортировка пузырьком исходной таблицы, сек. (%)	Быстрая сортировка исходной таблицы, сек. (%)	Сортировка пузырьком таблицы ключей, сек. (%)	Быстрая сортировка таблицы ключей, сек. (%)
50	0.000031	0.000008	0.000015	0.000006
	100%	380%	205%	538%
100	0.000091	0.000013	0.000043	0.000009
	100%	689%	210%	1058%
500	0.001513	0.000071	0.000659	0.000045
	100%	2119%	230%	3363%
1000	0.004497	0.000101	0.001722	0.000071
	100%	4453%	261%	6317%

По результатам замеров видно, что использование таблицы ключей выгоднее почти в 2 раза по времени при пузырьковой сортировке и примерно в 1,5 раз при быстрой сортировке.

Сравнение результатов по памяти:

Размер массива	Исходная таблица, байт	Таблица ключей, байт	Суммарная память, байт
50	4400	800	5200
100	8800	1600	10400
500	44000	8000	52000
1000	88000	16000	104000

7. Выводы

Итак, можно сделать вывод, что использование таблиц ключей при сортировке по одному полю массивов структур эффективнее в 2 раза при сортировке пузырьком и 1,5 раз при быстрой сортировке по времени, чем использование исходных таблиц, однако таблица ключей занимает дополнительную память в размере 18% от общего объёма.

Максимальная эффективность по времени получается при использовании таблицы ключей и быстрой сортировки в 538% при 50 элементов массива и в 6317% при 1000 элементов по сравнению с использованием пузырьковой сортировки исходной таблицы.

Оптимальный выбор между обработкой исходной таблицы и использованием ключей зависит от конкретной задачи: для небольших объемов данных или при ограничениях памяти предпочтительна работа с исходной таблицей, тогда как для больших массивов и частых перестановок эффективнее таблица ключей.

8. Ответы на контрольные вопросы

1. Как выделяется память под вариантную часть записи?

Память под вариантную часть записи выделяется статически, исходя из наибольшего возможного размера поля, описанного в этой записи. Даже если на практике используется другое поле вариантной части, резервируется память под всю возможную структуру, чтобы учесть наибольший возможный размер данных.

2. Что будет, если в вариантную часть ввести данные, несоответствующие описанным?

При вводе данных, не соответствующих описанным в вариантной части, произойдет некорректная интерпретация данных в памяти. Компилятор не может проверить корректность использования вариантной части во время выполнения, поэтому ответственность за правильность лежит на программисте.

3. Кто должен следить за правильностью выполнения операций с вариантной частью записи?

Ответственность за правильность выполнения операций с вариантной частью записи полностью лежит на программисте. Это может быть реализовано через:

- Проверку значения поля перед доступом к вариантной части;
- Валидацию вводимых данных при загрузке из файла и ручном добавлении;
- Использование перечисляемых типов для ограничения допустимых значений;
- Обработку ошибок при некорректном вводе.

4. Что представляет собой таблица ключей, зачем она нужна?

Таблица ключей представляет собой структуру данных, в которой хранятся ключи, позволяющие быстро находить записи в основной таблице. Ключи могут быть значениями определённых полей записи. Такая таблица используется для ускорения поиска и сортировки данных, а также для обеспечения целостности данных при работе с большими наборами информации.

5. В каких случаях эффективнее обрабатывать данные в самой таблице, а когда использовать таблицу ключей?

Оптимальный выбор между обработкой исходной таблицы и использованием ключей зависит от конкретной задачи: для небольших объемов данных или при ограничениях памяти предпочтительна работа с исходной таблицей, тогда как для больших массивов и частых перестановок эффективнее таблица ключей.

6. Какие способы сортировки предпочтительнее для обработки таблиц и почему?

Выбор метода сортировки зависит от конкретных характеристик данных и требований к производительности.

Быстрая сортировка (Quick Sort) — в худшем случае может иметь сложность $O(n^2)$, а в лучшем $O(n \cdot \log(n))$.

Пирамидальная сортировка (Heap Sort) – в худшем и лучшем случаях имеет сложность $O(n \cdot \log(n))$.